

Managing agent configuration using Git

Last Updated: 2026-04-20

You can manage agent configurations by using Git.

The Git-based configuration management functions are optional. If you already have other means of delivering file-based configurations for the host agent, |

Notes:

- The Git-based configuration management functions are available from the host agent bootstrap version 1.2.11.
- Host agent bootstrap version 1.2.12 adds support for HTTP/HTTPS basic authentication.

The host agent can retrieve its configurations from a remote Git repository. All the configurations that are discussed in the [host agent configuration](#) document effect (the documentation clarifies this requirement in the relevant section).

When to use Git-based configuration management [🔗](#)

The Instana host agent ships with default configurations, and you need not modify those configurations in most production scenarios.

However, features like [dynamic version pinning](#), the host agent [backend configurations](#) (especially when you use [multiple backends](#)), [secrets](#), and [host tags](#) to host agents that are deployed in entire landscapes or environments.

The Instana host agent has a built-in Git client that can fetch configurations from a remote repository that you operate. When the host agent discovers a *local* management capabilities, and looks for a [Git remote](#) in the local repository called `configuration`. On startup, the host agent updates its local repository to [Configuration updates](#) section.

The requirements of your Git repository setup are outlined in the [Prerequisites](#) section.

Prerequisites [🔗](#)

Note: The host agent configuration might include sensitive information (such as credentials), and should be protected from unauthorized access.

The Git-based configuration management does not assume any local Git client or other tools. The host agent needs access to the configured remote Git repository.

- The system that hosts the remote Git repository must be reachable by using a network connection from the host agent. The system needs to provide access. In addition, the appropriate ports must be open.
- Starting with agent bootstrap above version 1.2.12, basic HTTP/HTTPS authentication is supported. The environment variables `INSTANA_GIT_REMOTE`

Setup [🔗](#)

Self-signed certificates [🔗](#)

If you host the Git repository yourself, you can use a self-signed certificate. For the agent to connect securely to a Git repository, you must import the self-signed

Authentication with `.netrc` file [🔗](#)

Access to a Git repository can be authorized by using a `.netrc` file to provide the password or access token. `.netrc` is widely used and details about it can

You can configure the `.netrc` file in the home directory of the user (for example, `root`) where the Instana host agent is running. See the following sample `.netrc`

```
machine github.com
login oauth2
password <access_token>
```

Note: Replace `<access_token>` with an access token.

Git repository structure [🔗](#)

In the Git repository, you need to have same structure of directories as in `*instanaAgentDir*/etc`. For example, if you want to get data from the Git file `instana/configuration-mysql.yaml`). Otherwise, when the file is obtained from the Git repository, the file is placed in the wrong location on the server.

The following figure shows the example structure of a Git repository:

```
repository-root/
├── instana/
│   ├── configuration-mysql.yaml
│   ├── com.instana.agent.main.config.UpdateManager.cfg
│   └── org.ops4j.pax.url.mvn.cfg
```

With systemd [🔗](#)

By using systemd, the easiest way to specify the additional parameters is using a drop-in file. To view the path to the drop-in directory, run the following command:

```
systemctl status instana-agent
```

The files in this directory must have the file extension `.conf`, otherwise the files are ignored.

1. Create a new file in the `/etc/systemd/system/instana-agent.service.d/` directory with the extension `.conf`, such as `git-configuration-mysql.conf`.
2. Add the following content to the file:

```
[Service]
Environment=INSTANA_GIT_REMOTE_BRANCH=<branch>
Environment=INSTANA_GIT_REMOTE_REPOSITORY=<repository_url>
Environment=INSTANA_GIT_REMOTE_USERNAME=<username>
Environment=INSTANA_GIT_REMOTE_PASSWORD=<access_token>
...
**Notes:**
- Replace *<branch>* with the name of the remote branch that the Instana host agent connects to.
- Replace *<repository_url>* with the URL of the remote Git repository; for example, 'https://github.com/<your_account>/<your_repo>.git'.
- Replace *<username>* with a username or access token (for basic authentication). If you are using a '.netrc' file or a public repository, you can use the username or access token.
- Replace *<access_token>* with an access token. If you are using an access token as username (basic authentication), a '.netrc' file, or a public repository, you can use the access token.
```

After you add or change a drop-in file, complete the following steps:

1. Reload the drop-in files, by running the **systemctl daemon-reload** command.
2. Restart the Instana host agent, by running the command **systemctl restart instana-agent**.

Note: The agent initializes the local Git repository with the provided environment variables and will not change the remote configuration, even if the configuration is changed in the [Agent management dashboard](#) or by using the [Instana REST API](#). Alternatively, you can remove the `.git` folder in `*instanaAgentDir*/etc/`.

With environment variables [🔗](#)

If you have set the following environment variables, then when a host agent is started it checks for the presence of the local Git repository in the `*instanaAgentDir*/etc` directory and connects to the remote repository.

If the Instana agent is running directly on a host, the following environment variables need to be set on the host. If the host agent is running in a containerized environment, the environment variables need to be set on the container.

Environment variable	Description

Environment variable	Description
INSTANA_GIT_REMOTE_REPOSITORY	The URL of the remote Git repository; for example, <code>https://github.com/<you></code>
INSTANA_GIT_REMOTE_BRANCH	The name of the remote branch that the Instana host agent is to follow; for example, <code>main</code>
INSTANA_GIT_REMOTE_USERNAME	Optional: The username or access token to be used in basic authentication (See the note after this table)
INSTANA_GIT_REMOTE_PASSWORD	Optional: The password to be used in basic authentication (See the note after this table)

Note: If you want to use basic authentication with an access token instead of a username and password, set the access token in the parameter `INSTANA_GIT_REMOTE_USERNAME`.

Note: The agent initializes the local Git repository with the provided environment variables and will not change the remote configuration, even if the [Agent management dashboard](#) or by using the [Instana REST API](#). Alternatively, you can remove the `.git` folder in `*instanaAgentDir*/etc/`

With the one-liner command [↗](#)

Refer to the [-g, -b, -u and -p options](#) in the OneLiner documentation, which map to the `INSTANA_GIT_REMOTE_REPOSITORY`, `INSTANA_GIT_REMOTE_BRANCH`, `INSTANA_GIT_REMOTE_USERNAME`, and `INSTANA_GIT_REMOTE_PASSWORD` environment method.

The following table shows the mapping between the environment variables and the parameters of the one-liner command:

Environment variable	Parameter of the one-liner command
INSTANA_GIT_REMOTE_REPOSITORY	<code>-g</code> = (optional, required)
INSTANA_GIT_REMOTE_BRANCH	<code>-b</code> = (optional, required)
INSTANA_GIT_REMOTE_USERNAME	<code>-u</code> = (optional, required)
INSTANA_GIT_REMOTE_PASSWORD	<code>-p</code> = (optional, required)

Note: The agent initializes the local Git repository with the provided environment variables and will not change the remote configuration, even if the [Agent management dashboard](#) or by using the [Instana REST API](#). Alternatively, you can remove the `.git` folder in `*instanaAgentDir*/etc/`

With Docker images [↗](#)

The steps are fundamentally the same as the [environment method](#), by adding the `INSTANA_GIT_REMOTE_REPOSITORY`, `INSTANA_GIT_REMOTE_BRANCH`, `INSTANA_GIT_REMOTE_USERNAME`, and `INSTANA_GIT_REMOTE_PASSWORD` container.

For example, the following code sample gives the parameters that are needed to start the host agent in a container and the required Git parameters to enable the agent.

```
# Please enter proper values for all --env arguments.
docker run \
  --detach \
  --name instana-agent \
  --volume /var/run:/var/run \
  --volume /run:/run \
  --volume /dev:/dev:ro \
  --volume /sys:/sys:ro \
  --volume /var/log:/var/log:ro \
  --privileged \
  --net=host \
  --pid=host \
  --env="INSTANA_AGENT_ENDPOINT=" \
  --env="INSTANA_AGENT_ENDPOINT_PORT=" \
  --env="INSTANA_AGENT_KEY=" \
```

```
--env="INSTANA_DOWNLOAD_KEY=" \
--env="INSTANA_AGENT_ZONE=" \
--env="INSTANA_GIT_REMOTE_BRANCH=" \
--env="INSTANA_GIT_REMOTE_PASSWORD=" \
--env="INSTANA_GIT_REMOTE_REPOSITORY=" \
--env="INSTANA_GIT_REMOTE_USERNAME=" \
icr.io/instana/agent
```

With the agent management dashboard

The Git-based configuration management can be set up in the [Configuration management](#) section on the [Agent management dashboard](#).

! **Important:** You must add the `.git` suffix to the end of the `INSTANA_GIT_REMOTE_REPOSITORY` address, such as `https://github.com/<your-org>/<repo>.git`. You can use a `.netrc` file. For more information, see [Authentication with .netrc file](#).

With the API

The Git-based configuration management can be initialized and configured by using the API as described in the Host Agent section of the [Web REST API c](#)

Manual setup

A manual setup consists of initializing a Git repository in the `*instanaAgentDir*/etc/` folder and adding a remote named `configuration`.

The host agent pulls the local `main` branch by using the configured tracking branch from the `configuration` remote. There are multiple ways to set up the

For example, the following setup allows you to have the host agent pull configurations from the `main` branch of the `configuration` remote:

```
etc> git init
etc> git remote add -m main -t main configuration <git-repository-url>
etc> git fetch configuration
etc> git checkout -f -b main --track configuration/main
```

The following example allows you to pull the configurations from the `my-configurations` branch in the `configuration` remote:

```
etc> git init
etc> git remote add -m main -t my-configurations configuration <git-repository-url>
etc> git fetch configuration
etc> git checkout -f -b main --track configuration/my-configurations
```

Possible `<git-repository-url>` formats are described in the official documentation on [Git URLs](#).

Configuration updates

The host agent fetches configuration updates from the remote repository:

1. Upon the startup or restart.
2. Through a reboot initiated over the [Agent Management Dashboard](#).
3. Through a configuration change over the [Agent Management Dashboard](#).
4. Through the [Web API](#) as described in the Host Agent section and the integrations that rely on it like the [GitHub Update Agent action](#).

The Git-based configuration management operates on the local `main` branch, and updates it with the version of the configured remote tracking branch. If no update is run, the agent resumes its monitoring by using the current configurations.

Local changes are not expected and are discarded on updates. Untracked files are touched by the Git-based configuration management so that initially not c

Triggering updates by using Git Hooks [🔗](#)

Git Hooks are programs that you can place in the `.git/hooks` directory of the repository, and they trigger at different times in the lifecycle of your Git repository-based configuration management of host agents.

The [GitOps-Demo repository](#) provides a [sample post-update Git hook](#) that triggers the update of host agents whenever a branch in the repository is updated.

You see that the expected output of the script sent back to your Git client after you trigger a push:

```
$ gitops-test# git push origin HEAD:main
[detached HEAD 9632c09] origin
1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 376 bytes | 376.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0)
remote: GitOps update: Triggering the configuration update of agents matching the following query: 'entity.zone:"test"' ... OK
699ada9..9632c09 HEAD -> main
```

Triggering updates by using GitHub Update Action [🔗](#)

The [Instana GitHub action for GitOps](#) allows you to straightforwardly trigger the remote of the host agents by using Instana's Web API.

The [GitOps-Demo repository](#) shows how to set up a repository on GitHub to manage your host agent configurations and how to automatically trigger the updates.

Example [🔗](#)

Private GitHub repository [🔗](#)

GitHub supports HTTPS authentication by using tokens, and this can be used by the host agent as described in this section.

To use a GitHub repository, you first must create a personal access token. Details on how to do that are documented on the [Creating a personal access token](#).

GitHub expects the token to be provided as [HTTPS basic authentication username with an empty password](#). Therefore, the host agent can be configured to use the token as follows:

```
INSTANA_GIT_REMOTE_REPOSITORY='https://github.com/<user/organization>/<repository>.git'
INSTANA_GIT_REMOTE_BRANCH='<branch>'
INSTANA_GIT_REMOTE_USERNAME='<token>'
INSTANA_GIT_REMOTE_PASSWORD=''
```

Notice that the same principle of empty password and personal token as username works also with the other means of configuring the host agent for Git-based configuration management.

Controlling agent updates with Git [🔗](#)

You can use multiple methods to control the version updates to the host agent that is deployed into your production environments. If you use static agents, set update intervals, pinned versions, and so on. For more information, see [Configuring updates of dynamic host agents](#). If no configuration is specified, update version updates to the host agent, you can use a combination of dynamic agent version pinning with Git-based configuration management. If you have a test environment, you can use it to test updates before moving them into production.

Controlling Kubernetes agent updates

You can use Argo CD to control and manage the updates to the Kubernetes dynamic agents. By using this approach, you can specify the version of the agent that is used.

You can set the environment variable by using a custom resource for operator installations and the `values.yaml` file for Helm chart installations.

Argo CD monitors designated branches of the repository and automatically applies any new changes to the cluster. By using this approach to configure the updates, you can avoid updates during peak hours instead of running them automatically. Additionally, this setup provides you the flexibility to specify the exact agent version in use.

For an example and instructions on how to implement the workflow in which agent version pinning is combined with Argo CD integration, see the [instana/argo](#)

In the example, the repository contains the `main` branch for the pinning of the version in production clusters and the `test` branch for the pinning of the version. You can test any new versions in test clusters before promoting the agent version to production clusters. For information on how to use Mend Renovate to automate this section.

Controlling host agent updates

For an example on how to implement the workflow in which agent version pinning is combined with Git-based configuration management, see the [instana/git](#)

This example repository uses GitHub as a repository hosting service, Mend Renovate for dependency update automation, and GitHub Actions for sending requests to the Instana backend. To meet the following requirements to implement a Git-based update flow:

- Git as source code repository format
- automation to send an HTTP request to the Instana backend on configuration changes

You can use any automation of your choice here. However, make sure that the version in `instana/com.instana.agent.bootstrap.AgentBootstrap.cfg`

The repository contains the `main` branch that tracks the current state of the agent production deployments and the `test` branch that represents the agent test environment.

Mend Renovate is configured to watch the [instana/agent-updates](#) repository for new tags and compare the tags with the version in the `instana/com.instana.agent.bootstrap.AgentBootstrap.cfg` file. When a new tag is found in the `instana/agent-updates` repository, Mend Renovate creates a branch based on the `test` branch and raises a GitHub pull request to add the changes to the `test` branch. You can configure the pull request to [schedule updates only once a week](#), or to [assign certain GitHub handles to the pull requests](#) for manual approval. When a pull request is merged to the `test` branch, GitHub Actions runs a workflow that sends out an agent reboot command to all agents that match the particular zone and tag constraints of the test environment.

Additionally, a second workflow copies the current `instana/com.instana.agent.bootstrap.AgentBootstrap.cfg` file from the `test` branch to a new branch. When a pull request is merged to the `test` branch, a workflow sends out an agent reboot command to all agents that match the particular zone and tag constraints of the test environment to production.

After the changes are validated in a production-like environment, you can merge the changes to the `main` branch. When you merge, GitHub Actions runs a workflow that sends out an agent reboot command to all agents that match the particular zone and tag constraints of the production environment.

See the following sample configuration:

```
# This field specifies which version set of components should be used by the agent;
# its value must be a valid tag from https://github.com/instana/agent-updates/tags
# version = <tag>
# renovate: datasource=github-tags depName=instana/agent-updates versioning=loose
version = 2024.09.02.0634

# This field controls which set of components is used by the agent at runtime.
productName = ${env:INSTANA_PRODUCT_NAME}

origin = package dynamic
```

Note: Whenever the Instana backend receives an agent configuration update, it restarts all matching agents. Depending on the number of agents and the frequency of updates, over time, you can extend the example with additional branches that represent batches of agents to update. You can also adjust the pre-defined delay to avoid increased manual management effort.